Question 1 (Easy):

A university's Training and Placement Cell have successfully completed its annual placement drive. The placement department has collected the salary packages (in LPA) offered to students by different companies. However, the records were entered in the order they were received, resulting in an unsorted list of package values.

Before publishing the placement statistics report, the department wants all package values arranged in ascending order. Since the university plans to handle placement data for thousands of students in the coming years, they want a sorting technique that is efficient for large datasets.

Your task is to implement the **Merge Sort algorithm** to sort the package values in increasing order. Merge Sort follows a divide-and-conquer approach by dividing the dataset into smaller subarrays, sorting them recursively, and then merging them back together.

The final sorted list will help the university identify minimum, median, and maximum packages while generating analytical reports.

Input

- First line contains an integer **N**, representing the number of placed students.
- Second line contains **N space-separated integers**, representing package values (multiplied by 100 to avoid decimals).

Output

Print the package values in ascending order.

Constraints

- $1 \leq N \leq 100000$
- $100 \leq \text{Package} \leq 10000$

Example

Input:

6
450 1200 800 650 300 1500

Output:

300 450 650 800 1200 1500

WEEK-3 x MODULE

Merge Sort Algorithm

Q) Input :- 6
450 1200 800 650 300 1500

Output :- 300 450 650 800 1200 1500

Logic :- i) Read N & the array elements.
ii) Divide the array into two halves recursively until each subarray has one element.
iii) Merge the sorted halves by comparing elements & placing them in ascending order.
iv) After the complete merge process, print the sorted array.

Time complexity : O(N log N)
Space Complexity : O(N)

Code :

```
#include <iostream>
#include <vector>
using namespace std;
void merge(vector<int> &a, int l, int m, int r){
vector<int> t;
int i=l, j=m+1;
while(i<=m && j<=r){
if(a[i]<=a[j]) t.push_back(a[i++]);
else t.push_back(a[j++]);
}
while(i<=m) t.push_back(a[i++]);
while(j<=r) t.push_back(a[j++]);
for(int k=0; k<t.size(); k++)
a[l+k]=t[k];
}
void mergesort(vector<int> &a, int l, int r){
if(l>=r) return;
int m=(l+r)/2;
mergesort(a, l, m);
mergesort(a, m+1, r);
merge(a, l, m, r);
}
int main(){
int n;
cin>>n;
```

```
vector<int> a(n);
for(int i=0; i<n; i++)
  cin>>a[i];
mergesort(a, 0, n-1);
for(int i=0; i<n; i++)
  cout<<a[i]<<" ";
return 0;
}
```

Output

```
6

450 1200 800 650 300 1500
300 450 650 800 1200 1500

=== Code Execution Successful ===
```

Question 2 (Medium):

A large e-commerce company tracks the time taken (in minutes) to process customer orders before shipment. Due to varying warehouse workloads, processing times differ significantly from one order to another.

The analytics team wants to study operational efficiency by sorting all processing times and generating statistical information. Since millions of orders are processed every month, the company requires an efficient sorting technique with predictable performance.

Your task is to use **Merge Sort** to arrange all processing times in ascending order.
After sorting, perform the following operations:

1. Display the sorted processing times.
2. Find the **median processing time**.
3. Count the number of orders whose processing time is greater than the median.
4. Display the difference between the fastest and slowest processing time.

The solution should efficiently handle large datasets and demonstrate a clear understanding of Merge Sort's divide-and-conquer strategy.

Input

- First line contains integer **N**.
- Second line contains **N space-separated processing times**.

Output

- Sorted processing times.
- Median processing time.
- Count of orders greater than median.
- Difference between maximum and minimum processing time.

Constraints

- $1 \leq N \leq 100000$
- $1 \leq Time \leq 100000$

Example

Input:

7
12 25 18 30 15 22 40

Output:

12 15 18 22 25 30 40
Median: 22
Orders Above Median: 3
Difference: 28

MERGE SORT ALGORITHM

Q.)2) Input :- 7
12 25 18 30 15 22 40

output :- 12 15 18 22 25 30 40
Median : 22
Orders Above Median : 3
Difference : 28.

Logic :- i) Merge sort :- Split array in half, sort recursively & merge back together.

ii) Median :- Pick the middle number if odd, or average the two middle numbers if even.

iii) Count :- Loop through the sorted numbers & count how many are greater than two medians.

iv) Difference :- Subtract the first number (minm) from the last number (maxm)

Code :-

```
#include <iostream>
# include <vector>
using namespace std;
void merge(vector<int>& arr, int left, int mid, int
                    right) {
int n1 = mid - left + 1;
int n2 = right - mid;
vector<int> L(n1), R(n2);
for(int i=0; i<n1; i++) L[i] = arr[left+i];
for(int j =0; j<n2; j++) R[j] = arr[mid+1+j];
int i=0, j=0, k=left;
while(i<n1 && j<n2) {
   if(L[i]<=R[j]) {
```

```
        arr[k] = L[i];
        i++;
    } else {
        arr[k] = R[j];
        j++;
    }
    k++;
}
while(i < n1) {
    arr[k] = L[i];
    i++;
    k++;
}
while(j < n2) {
    arr[k] = R[j];
    j++;
    k++;
}
}
void mergesort(vector<int>& arr, int left, int right) {
if(left < right) {
    int mid = left + (right - left) / 2;
mergesort(arr, left, mid);
mergesort(arr, mid+1, right);
merge(arr, left, mid, right);
}
int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    int n;
    if(!(cin >> n)) return 0;
vector<int> times(n);
for(int i=0; i<n; i++) {
    cin >> times[i];
}
mergesort(times, 0, n-1);
```

```
for(int i=0 ; i<n; i++) {
  cout << times[i] << (i==n-1?"":" ");
} cout << "\n";
double median;
if (n%2==1) {
  median = times[n/2];
} else {
  median = (times[(n/2)-1] + times[n/2])/2.0;
}
cout << "Median: " << median << "\n";
int count_above = 0;
for (int i=0; i<n; i++) {
  if (times[i] > median) {
    count_above++;
  }
}
cout << "orders above Median: " << count_above
  << "\n";
int difference = times[n-1] - times;
cout << "Difference: " << difference << "\n";
  Return 0;
}
```

Output

```
7
12 25 18 30 15 22 40
12 15 18 22 25 30 40
Median: 22
Orders Above Median: 3
Difference: 28


=== Code Execution Successful ===
```

Topic 2: Quick Sort

Question 1 (Easy):

A cloud infrastructure company manages hundreds of servers distributed across different geographical locations. Every hour, each server reports its average response time in milliseconds. The collected data is stored in the order it arrives and is not sorted.

To generate performance reports, the operations team wants the response times arranged from the fastest server to the slowest server. Since the data size can become very large, an efficient in-place sorting technique is required.

Your task is to implement **Quick Sort** to sort the response times in ascending order. Built-in sorting methods are not allowed.

After sorting, the operations team will use the report to identify high-latency servers and optimize system performance.

Input

- First line contains integer **N**.
- Second line contains **N space-separated response times**.

Output

Print the response times in ascending order.

Constraints

- $1 \leq N \leq 100000$
- $1 \leq$ Response Time ≤ 10000

Example

Input:

6
120 45 78 200 60 95

Output:

45 60 78 95 120 200

Logic (Short) :-

1) Read N and the array elements.
2) Select the last element as the pivot.
3) Partition the array so smaller elements come before the pivot and larger elements after it.
4) Recursively apply Quick Sort on the left and right subarrays.
5) Print the sorted array.

Time Complexity : $O(N \log N)$ (average),
$O(N^2)$ (worst case)

Space Complexity : $O(\log N)$ (recursive stack)

```
#include <iostream>
#include <vector>
using namespace std;

int partition(vector<int> &a, int low, int high) {
    int pivot = a[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (a[j] <= pivot) {
            i++;
            swap(a[i], a[j]);
        }
    }
    swap(a[i+1], a[high]);
    return i + 1;
}
void quickSort(vector<int> &a, int low, int high) {
    if (low < high) {
        int pi = partition(a, low, high);
        quickSort(a, low, pi - 1);
        quickSort(a, pi + 1, high);
    }
}
int main() {
    int n;
    cin >> n;
    vector<int> a(n);
    for (int i = 0; i < n; i++)
        cin >> a[i];
    quickSort(a, 0, n - 1);
    for (int i = 0; i < n; i++) {
        cout << a[i];
        if (i != n - 1) cout << " ";
    }
    return 0;
}
```

Output

```
6

120 45 78 200 60 95
45 60 78 95 120 200

=== Code Execution Successful ===
```

Question 2 (Medium):

A financial analytics company receives daily trade-value data from multiple stock exchanges. Analysts use this information to identify high-value trading activity and market trends.

The trade values are received in random order throughout the day. Before analysis can begin, the data must be sorted in descending order.

Your task is to implement **Quick Sort** to arrange the trade values from highest to lowest.

After sorting, perform the following operations:

1. Display the sorted trade values.
2. Find the **Top 5 highest trade values**.
3. Calculate the average of the Top 5 values.
4. Count how many trade values are greater than the overall average of all trade values.

This problem evaluates your understanding of partitioning, recursion, and real-world data analysis using Quick Sort.

Input

- First line contains integer **N**.
- Second line contains **N space-separated trade values**.

Output

- Sorted trade values (descending).
- Top 5 values.
- Average of Top 5 values.
- Count of values greater than overall average.

Constraints

- $5 \leq N \leq 100000$
- $1 \leq$ Trade Value $\leq 10^9$

Example

Input:

8
500 1200 700 2000 900 1500 3000 2500

Output:

3000 2500 2000 1500 1200 900 700 500
Top 5: 3000 2500 2000 1500 1200
Average of Top 5: 2040
Values Above Overall Average: 4

Logic :-

1) Read N and the trade values.

2) Apply Quick Sort in descending order using the last element as the pivot.

3) Print the sorted array.

4) Find the Top 5 values and calculate their average.

5) Calculate the overall average of all values and count how many values are greater than it.

Time Complexity : $O(N \log N)$ (average),
$O(N^2)$ (worst case)

Space Complexity : $O(\log N)$ (recursive stack)

```
#include <iostream>
#include <vector>
using namespace std;

int partition (vector < long long> & a , int low, int high) {
      long long pivot = a [high];
      int i = low - 1;
      for (int j = low; j < high; j++) {
            if (a [j] >= pivot) {
                  i++;
                  swap (a [i], a [j]);
            }
      }
      swap (a [i+1], a [high]);
      return i + 1;
  }
  void quickSort (vector < long long> & a , int low, int high) {
      if (low < high) {
            int pi = partition (a, low, high);
            quickSort (a, low, pi - 1);
            quickSort ( a, pi + 1, high);
      }
  }
  int main ( ) {
      int n;
      cin >> n;
      vector < long long> a (n);
      long long sum = 0;
      for (int i = 0; i < n; i++) {
            cin >> a [i];
            sum += a [i];
      }
      quickSort (a , 0 , n - 1);
      for (int i = 0; i < n; i++)
            cout << a [i] << " ";
      cout << endl;
      cout << " Top 5 : ";
      long long topSum = 0;
      for (int i = 0; i < 5; i++) {
            cout << a [i] << " ";
            topSum += a [i];
      }
      cout << endl;
      cout << " Average of Top 5 : " << topSum / 5 << endl;
      double avg = (double) sum / n;
      int count = 0;
      for (int i = 0; i < n; i++)
            if (a [i] > avg)
                  count ++;
      cout << " Values Above Overall Average : " << count;
      return 0;
}
```

Output

```
8

500 1200 700 2000 900 1500 3000 2500
3000 2500 2000 1500 1200 900 700 500
Top 5: 3000 2500 2000 1500 1200
Average of Top 5: 2040
Values Above Overall Average: 3

=== Code Execution Successful ===
```

Topic 3: Heap Sort

Question 1 (Easy):

An online gaming platform is hosting a national-level tournament involving thousands of participants. At the end of the tournament, player scores are collected from multiple game servers and stored in random order.

Before announcing the final rankings, the organizers need all scores sorted in ascending order. Since the leaderboard data can be very large, they have decided to use **Heap Sort**, which provides guaranteed O(N log N) performance.

Your task is to implement Heap Sort and arrange all player scores in increasing order.

The sorted output will help tournament officials generate rankings and determine qualification thresholds for future events.

Input

- First line contains integer **N**.
- Second line contains **N space-separated player scores**.

Output

Print the scores in ascending order.

Constraints

- $1 \leq N \leq 100000$
- $0 \leq Score \leq 1000000$

Example

Input:

6
850 1200 950 600 1500 1000

Output:

600 850 950 1000 1200 1500

```
#include <iostream>
#include <vector>
using namespace std;
void heapify(vector<int> &a, int n, int i)
{
    int largest = i;
    int left = 2*i+1;
    int right = 2*i+2;
    if (left < n && a[left] > a[largest])
        largest = left;
    if (right < n && a[right] > a[largest])
        largest = right;
    if (largest != i)
    {
        swap(a[i], a[largest]);
        heapify(a, n, largest);
    }
}
void heapSort(vector<int> &a, int n)
{
    for (int i = n/2-1; i >= 0; i--)
        heapify(a, n, i);
    for (int i = n-1; i > 0; i--)
    {
        swap(a[0], a[i]);
        heapify(a, i, 0);
    }
}
int main()
{
    int n;
    cin >> n;
    vector<int> a(n);
    for (int i = 0; i < n; i++)
        cin >> a[i];
    heapSort(a, n);
    for (int i = 0; i < n; i++)
        cout << a[i] << " ";
    return 0;
}
```

Heap Sort Logic :-

1) Read the array elements.

2) Convert the array into a Max Heap.

3) Swap the root (largest element) with the last element.

4) Reduce the heap size by 1.

5) Apply heapify() on the root to restore the Max Heap.

6) Repeat steps 3-5 until only one element remains.

7) The array is now sorted in ascending order.

Time Complexity : $O(N \log N)$

Space Complexity : $O(1)$

Output

```
Input:

6

850 1200 950 600 1500 1000


=== Code Execution Successful ===
```

Question 2 (Medium):

A multi-specialty hospital maintains records of emergency response times for all critical cases handled during a month. Each response time represents the number of minutes taken by the emergency team to reach and begin treatment for a patient.

Hospital administrators want to analyze the efficiency of their emergency services. To do so, all response times must first be sorted using **Heap Sort**.

After sorting the response times in ascending order, generate the following statistics:

1. Display the sorted response times.
2. Find the fastest response time.
3. Find the slowest response time.
4. Calculate the average response time.
5. Count the number of cases handled faster than the average.
6. Calculate the percentage of cases handled faster than the average.

The hospital intends to use this information to improve resource allocation and reduce emergency response delays.

Input

- First line contains integer **N**.
- Second line contains **N space-separated response times**.

Output

- Sorted response times.
- Fastest response time.
- Slowest response time.
- Average response time.
- Number of cases faster than average.
- Percentage of cases faster than average.

Constraints

- $1 \leq N \leq 100000$
- $1 \leq Time \leq 500$

Example

Input:

8
12 7 15 9 20 5 10 8

Output:

5 7 8 9 10 12 15 20
Fastest: 5
Slowest: 20
Average: 10.75
Cases Faster Than Average: 5
Percentage: 62.50%

Logic :-

1) Read N and the response times.

2) Sort the array using Heap Sort.

3) Print the sorted array.

4) Find fastest = first element and slowest = last element.

5) Calculate average = sum of all elements ÷ N.

6) Count elements smaller than the average.

7) Calculate percentage = (count × 100.0) / N.

8) Print fastest, slowest, average, count, and percentage.

```cpp
#include <iostream>
#include <vector>
#include <iomanip>
using namespace std;

void heapify (vector<int> &a, int n, int i)
{
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    if (left < n && a[left] > a[largest])
        largest = left;
    if (right < n && a[right] > a[largest])
        largest = right;
    if (largest != i)
    {
        swap (a[i], a[largest]);
        heapify (a, n, largest);
    }
}
void heapSort (vector<int> &a, int n)
{
    for (int i = n/2 - 1; i >= 0; i--)
        heapify (a, n, i);
    for (int i = n-1; i > 0; i--)
    {
        swap (a[0], a[i]);
        heapify (a, i, 0);
    }
}
int main ()
{
    int n;
    cin >> n;
    vector<int> a(n);
    double sum = 0;
    for (int i=0; i<n; i++)
    {
        cin >> a[i];
        sum += a[i];
    }
    heapSort (a, n);
    for (int i=0; i<n; i++)
        cout << a[i] << " ";
    cout << endl;
    int fastest = a[0];
    int slowest = a[n-1];
    double avg = sum / n;
    int count = 0;
    for (int i=0; i<n; i++)
        if (a[i] < avg)
            count++;
    double percent = (count * 100.0) / n;
    cout << "Fastest: " << fastest << endl;
    cout << "Slowest: " << slowest << endl;
    cout << fixed << setprecision(2);
    cout << "Average: " << avg << endl;
    cout << "Cases Faster Than Average: " << count << endl;
    cout << "Percentage: " << percent << "%" << endl;
    return 0;
}
```

Output

```
8

12 7 15 9 20 5 10 8
5 7 8 9 10 12 15 20
Fastest: 5
Slowest: 20
Average: 10.75
Cases Faster Than Average: 5
Percentage: 62.50%


=== Code Execution Successful ===
```